transactions

Remy Wang

April 2026

So far we've been treating DBs as inanimate objects: they hold data, we can look at the data, and that's it. But SQL databases are so popular because they deal with *changes* very well. Let's see some examples.

I'm in a good mood today and decided to give everyone free points! I vibed a quick python script to do this:

UID	score
1	39
2	24
3	40
4	35
...	...

I'm in a good mood today and decided to give everyone free points! I vibed a quick python script to do this:

UID	score
1	39
2	24
3	40
4	35
...	...

Uh oh! My computer crashed halfway through, and I don't know who still haven't got the free points!

You don't like that, because some of you got the points and some didn't. You would be fine if either none of you get the points or all of you did.

Let's try another experiment. This time, your fate is in your own hands: steal points from your enemies! Subtract some points from someone and add it to your own. Here's the Google sheet, go!

Well, that was a mess. The total score no longer add up, meaning our DB is not *consistent*!

The problem is that you were *interfering* with each other!

Well, that was a mess. The total score no longer add up, meaning our DB is not *consistent*!

The problem is that you were *interfering* with each other!

One solution is to go *one at a time*, so that every change is made in *isolation*. In other words, we *serialize* the changes.

Another solution is we first claim a *lock* on a piece of data before changing it. This time, wait a few seconds before editing a cell, and only edit it if no one else is trying to edit.

Formally, a group of actions on a DB is called a *transaction*. DBs like SQLite guarantee transactions (tx) are:

- **Atomic**: either all or none of the actions are applied (all or nothing)

Formally, a group of actions on a DB is called a *transaction*. DBs like SQLite guarantee transactions (tx) are:

- **Atomic**: either all or none of the actions are applied (all or nothing)
- **Consistent**: after a tx finishes, the DB is in a consistent state

Formally, a group of actions on a DB is called a *transaction*. DBs like SQLite guarantee transactions (tx) are:

- **Atomic**: either all or none of the actions are applied (all or nothing)
- **Consistent**: after a tx finishes, the DB is in a consistent state
- **Isolated**: txs do not interfere with each other

Formally, a group of actions on a DB is called a *transaction*. DBs like SQLite guarantee transactions (tx) are:

- **Atomic**: either all or none of the actions are applied (all or nothing)
- **Consistent**: after a tx finishes, the DB is in a consistent state
- **Isolated**: txs do not interfere with each other
- **Durable**: once a tx finishes, its effect is kept forever

Formally, a group of actions on a DB is called a *transaction*. DBs like SQLite guarantee transactions (tx) are:

- **Atomic**: either all or none of the actions are applied (all or nothing)
- **Consistent**: after a tx finishes, the DB is in a consistent state
- **Isolated**: txs do not interfere with each other
- **Durable**: once a tx finishes, its effect is kept forever

Formally, a group of actions on a DB is called a *transaction*. DBs like SQLite guarantee transactions (tx) are:

- **Atomic**: either all or none of the actions are applied (all or nothing)
- **Consistent**: after a tx finishes, the DB is in a consistent state
- **Isolated**: txs do not interfere with each other
- **Durable**: once a tx finishes, its effect is kept forever

This is known as *ACID* on the street.

Note that *consistent* is a high-level property, usually achieved with the other 3.

- When only some of you got the extra credit, our tx was not *atomic*
- When you were stealing points and had conflicts, you were not *isolated*, leaving the DB *inconsistent*
- We haven't see *durable* yet, but if the computer dies before writing data to disk, the data would be lost

- When only some of you got the extra credit, our tx was not *atomic*
- When you were stealing points and had conflicts, you were not *isolated*, leaving the DB *inconsistent*
- We haven't see *durable* yet, but if the computer dies before writing data to disk, the data would be lost

Luckily, the DB handles all these for us!

But it's still important to understand what's going on, in case anything goes sideways.

There are several different ways to ensure *atomicity*. The main idea is to a tx to be explicitly `COMMIT`ted:

```
BEGIN TRANSACTION;

SELECT ...;
INSERT ...;

COMMIT;
```

You can think of each tx as making a local copy of the DB, and any operation in that tx only acts upon the local copy, until the tx is committed which writes the DB to the global one.

Try running the following concurrently in 2 SQLite sessions[1]:

```
--      SESSION 1                       SESSION 2

create table r (x int);
insert into r values (1);
                                        select * from r;
begin transaction;
insert into r values (2);
                                        select * from r;
commit;
                                        select * from r;
```

You'll see the change made within the tx does not take effect until after `COMMIT`.

[1] Run `sqlite3 test.db` in 2 different terminal tabs.

The easiest way to guarantee *isolation* is to *serialize* the transactions – running them one at a time. This is pretty much what SQLite does![2] So why don't we just leave it at that?

[2] SQLite serializes writes, but allow concurrent reads.

The easiest way to guarantee *isolation* is to *serialize* the transactions – running them one at a time. This is pretty much what SQLite does![2] So why don't we just leave it at that?

Because performance. When we serialized our score updates, we all had to wait in line, even though we could have used some parallelism.

[2] SQLite serializes writes, but allow concurrent reads.

So on one hand, serialization guarantees consistency but is slow

On the other hand, concurrency is fast but is inconsistent

The key idea of TX processing is to preserve consistency while improving concurrency, which is usually done by giving the *illusion* of serial execution while running concurrently. Magic!

First off, if 2 TX touch different data, they can of course run concurrently:

T_1	T_2
R(A)	R(B)
W(A)	W(B)

Here R(A) means reading a DB item A, and W(A) writes to A. You can think of a DB item as a row, but it depends on the context.

The table here shows the *schedule*[3] of the txs which is a record of actions taken over time.

[3] Formally, a schedule is a *partial ordering* of the actions.

We say a schedule is *serial* if all actions are strictly ordered:

T_1	T_2
R(A)	
W(A)	
	R(B)
	W(B)

In our example, the concurrent schedule happens to be *equivalent* to the serial one, because the TX involve independent items:

T_1	T_2
R(A)	R(B)
W(A)	W(B)

T_1	T_2
R(A)	
W(A)	
	R(B)
	W(B)

Formally, two TX are *equivalent* if they produce the same result no matter what is read or written. . . . And we say the concurrent schedule is *serializable*, i.e., it is equivalent to some serial schedule.

The main idea of DBs is that we want to allow **serializable**, yet concurrent **transaction schedules** like the above.[4]

[4] Make sure you know what the bold words mean by now!

Note that two serial schedules are not necessarily equivalent:

T_1	T_2
A = 2*A	
	A = 2+A

T_1	T_2
	A = 2+A
A = 2*A	

If we start with A = 1, the first schedule would result in A = 4, while the second in A = 6.

Note that two serial schedules are not necessarily equivalent:

T_1	T_2
A = 2*A	
	A = 2+A

T_1	T_2
	A = 2+A
A = 2*A	

If we start with A = 1, the first schedule would result in A = 4, while the second in A = 6.

For a schedule to be *serializable*, it only needs to be equivalent to *some* serial one, but we don't know which one.

There's also the notion of *strict serializability*, which basically says "first come, first serve":

- A schedule is strict-serializable if it's equivalent to a serial schedule that runs the TXs in the same order they arrive.

This can be important for e.g. ticket sales.